

Untitled-1

December 12, 2025

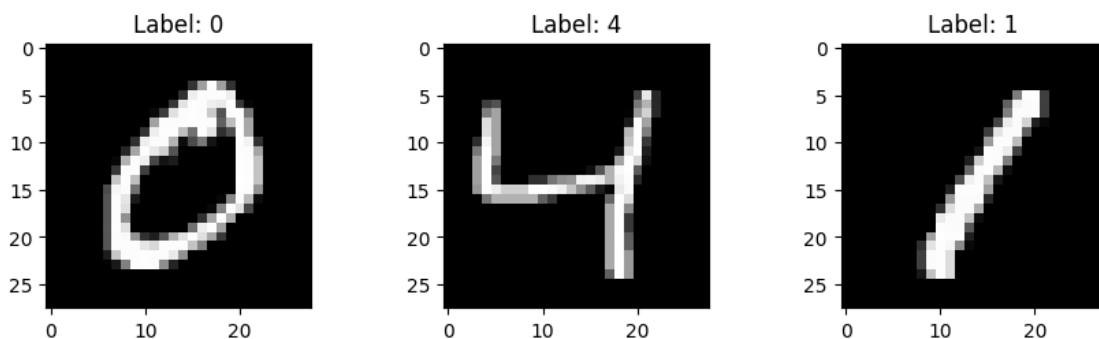
```
[1]: import numpy as np
import os
os.environ["KERAS_BACKEND"] = "torch"
from tensorflow import keras
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib.ticker import (MultipleLocator, FormatStrFormatter)
```

```
[2]: trainData = pd.read_csv("mnist_train.zip")
testData = pd.read_csv("mnist_test.zip")
Xtrain , Ytrain = trainData.iloc[:,1:].to_numpy(),trainData.iloc[:,0].to_numpy()
Xtest , Ytest = testData.iloc[:,1:].to_numpy(),testData.iloc[:,0].to_numpy()
```

```
[3]: trainData.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 59999 entries, 0 to 59998
Columns: 785 entries, 5 to 0.617
dtypes: int64(785)
memory usage: 359.3 MB
```

```
[4]: plt.figure(figsize=(10,10))
for i in range(3):
    plt.subplot(1, 3, i+1)
    plt.imshow(Xtrain[i].reshape(28,28), cmap='gray')
    plt.title(f"Label: {Ytrain[i]}")
    plt.axis(True)
    plt.subplots_adjust(wspace=0.5,hspace=0.5)
```



```
[5]: Xtrain = Xtrain.astype('float32')/255.0
Xtest = Xtest.astype('float32')/255.0
```

```
[6]: Ytrain = keras.utils.to_categorical(Ytrain, num_classes=10)
Ytest = keras.utils.to_categorical(Ytest, num_classes=10)
```

```
[7]: Ytrain[1]
```

```
[7]: array([0., 0., 0., 0., 1., 0., 0., 0., 0., 0.])
```

```
[8]: model = keras.Sequential([
    keras.layers.Input(shape=(Xtrain.shape[1],)),
    keras.layers.Dense(128, activation='relu'),
    keras.layers.Dense(128, activation='relu'),
    keras.layers.Dense(10, activation='softmax')
])
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 128)	100,480
dense_1 (Dense)	(None, 128)	16,512
dense_2 (Dense)	(None, 10)	1,290

Total params: 118,282 (462.04 KB)

Trainable params: 118,282 (462.04 KB)

Non-trainable params: 0 (0.00 B)

```
[9]: model.compile(optimizer=keras.optimizers.Adam(learning_rate=0.001),
    loss=keras.losses.CategoricalCrossentropy(),
    metrics=[keras.metrics.CategoricalAccuracy(name="accuracy")])
```

```
[10]: history = model.fit(Xtrain, Ytrain,
    validation_split=0.15,
```

```
batch_size=128,  
epochs=20,  
shuffle=True)
```

Epoch 1/20

399/399 16s 40ms/step -

accuracy: 0.9014 - loss: 0.3447 - val_accuracy: 0.9546 - val_loss: 0.1600

Epoch 2/20

399/399 16s 41ms/step -

accuracy: 0.9575 - loss: 0.1434 - val_accuracy: 0.9687 - val_loss: 0.1105

Epoch 3/20

399/399 17s 42ms/step -

accuracy: 0.9704 - loss: 0.0980 - val_accuracy: 0.9723 - val_loss: 0.0933

Epoch 4/20

399/399 16s 40ms/step -

accuracy: 0.9782 - loss: 0.0716 - val_accuracy: 0.9727 - val_loss: 0.0923

Epoch 5/20

399/399 17s 42ms/step -

accuracy: 0.9843 - loss: 0.0534 - val_accuracy: 0.9760 - val_loss: 0.0809

Epoch 6/20

399/399 17s 42ms/step -

accuracy: 0.9868 - loss: 0.0433 - val_accuracy: 0.9747 - val_loss: 0.0873

Epoch 7/20

399/399 17s 42ms/step -

accuracy: 0.9896 - loss: 0.0341 - val_accuracy: 0.9786 - val_loss: 0.0765

Epoch 8/20

399/399 16s 41ms/step -

accuracy: 0.9923 - loss: 0.0261 - val_accuracy: 0.9779 - val_loss: 0.0817

Epoch 9/20

399/399 16s 41ms/step -

accuracy: 0.9932 - loss: 0.0220 - val_accuracy: 0.9761 - val_loss: 0.0939

Epoch 10/20

399/399 16s 41ms/step -

accuracy: 0.9940 - loss: 0.0191 - val_accuracy: 0.9764 - val_loss: 0.0927

Epoch 11/20

399/399 17s 41ms/step -

accuracy: 0.9953 - loss: 0.0155 - val_accuracy: 0.9760 - val_loss: 0.0918

Epoch 12/20

399/399 17s 43ms/step -

accuracy: 0.9963 - loss: 0.0128 - val_accuracy: 0.9766 - val_loss: 0.0957

Epoch 13/20

399/399 16s 41ms/step -

accuracy: 0.9965 - loss: 0.0116 - val_accuracy: 0.9761 - val_loss: 0.0945

Epoch 14/20

399/399 17s 42ms/step -

accuracy: 0.9963 - loss: 0.0115 - val_accuracy: 0.9771 - val_loss: 0.1013

Epoch 15/20

399/399 16s 41ms/step -

```
accuracy: 0.9971 - loss: 0.0091 - val_accuracy: 0.9761 - val_loss: 0.1065
Epoch 16/20
399/399          17s 42ms/step -
accuracy: 0.9976 - loss: 0.0069 - val_accuracy: 0.9763 - val_loss: 0.1109
Epoch 17/20
399/399          16s 40ms/step -
accuracy: 0.9958 - loss: 0.0123 - val_accuracy: 0.9770 - val_loss: 0.1091
Epoch 18/20
399/399          16s 41ms/step -
accuracy: 0.9973 - loss: 0.0080 - val_accuracy: 0.9797 - val_loss: 0.1065
Epoch 19/20
399/399          16s 41ms/step -
accuracy: 0.9980 - loss: 0.0064 - val_accuracy: 0.9779 - val_loss: 0.1140
Epoch 20/20
399/399          16s 41ms/step -
accuracy: 0.9987 - loss: 0.0044 - val_accuracy: 0.9763 - val_loss: 0.1357
```

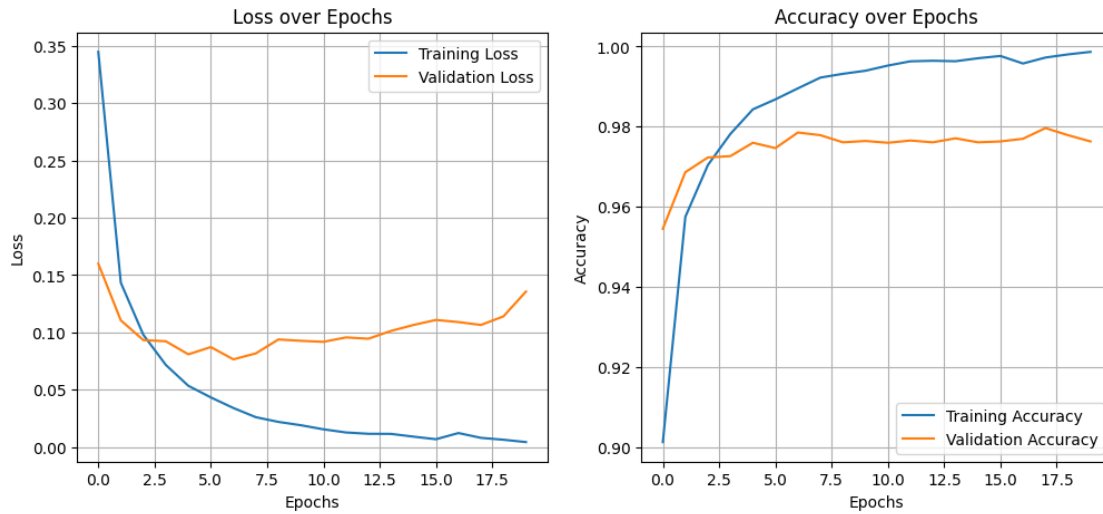
```
[11]: def plot_results(history):
        fig, ax = plt.subplots(1,2, figsize=(12,5))

        # Plot loss
        ax[0].plot(history.history['loss'], label='Training Loss')
        ax[0].plot(history.history['val_loss'], label='Validation Loss')
        ax[0].set_title('Loss over Epochs')
        ax[0].set_xlabel('Epochs')
        ax[0].set_ylabel('Loss')
        ax[0].legend()
        ax[0].grid(True)

        # Plot accuracy
        ax[1].plot(history.history['accuracy'], label='Training Accuracy')
        ax[1].plot(history.history['val_accuracy'], label='Validation Accuracy')
        ax[1].set_title('Accuracy over Epochs')
        ax[1].set_xlabel('Epochs')
        ax[1].set_ylabel('Accuracy')
        ax[1].legend()
        ax[1].grid(True)

        plt.show()
```

```
[12]: plot_results(history)
```



```
[13]: Ypred = model.predict(Xtest)
```

```
1/313          7s 24ms/step
```

```
313/313        3s 11ms/step
```

```
[14]: print("Predicted labels: ", np.argmax(Ypred, axis=1)[:10])
print("True labels:         ", np.argmax(Ytest, axis=1)[:10])
```

```
Predicted labels:  [2 1 0 4 1 4 9 6 9 0]
```

```
True labels:       [2 1 0 4 1 4 9 5 9 0]
```

```
[15]: print(Ytest[0])
print(Ypred[0])
```

```
[0. 0. 1. 0. 0. 0. 0. 0. 0. 0.]
```

```
[1.0579677e-13 1.9538978e-08 1.0000000e+00 4.3701065e-10 2.1128698e-22
```

```
5.5059905e-16 3.8273115e-16 2.5851791e-15 2.9726441e-13 2.2553844e-22]
```

```
[ ]:
```